



**CENTRO REGIONAL
UNIVERSITARIO
CÓRDOBA IUA**

**FACULTAD DE INGENIERÍA
CENTRO REGIONAL UNIVERSITARIO IUA**

Desarrollo de Herramientas de Software

INFORME TRABAJO FINAL

Autora:

NUÑEZ, Celeste Aylén

Profesor:

ESCHOYEZ, Maximiliano A.

1. Introducción y Problemática

Este informe describe el trabajo final de Desarrollo de Herramientas de Software: construir un compilador para un subconjunto reducido de C. El proyecto cubre las etapas típicas de un compilador: análisis léxico, sintáctico y semántico, generación de código intermedio y optimización.

El problema central es procesar un archivo fuente y determinar si es válido, tanto en su forma como en su significado. En lo sintáctico hay que controlar que los delimitadores estén balanceados, que las expresiones estén bien formadas y que cada instrucción termine en punto y coma. En lo semántico, el desafío principal es manejar distintos contextos mediante una tabla de símbolos, y verificar que las llamadas a funciones y las sentencias return sean coherentes con la forma en que cada función fue definida.

Si el archivo tiene errores, el compilador los informa y no continúa. Si es válido, genera Código de Tres Direcciones y lo optimiza, sin cambiar el comportamiento del programa original.

2. Desarrollo de la Solución Propuesta

La solución se implementó en Python 3, usando ANTLR4 para generar el lexer y el parser a partir de una gramática propia (compilador.g4), Maven para las dependencias y Git para el trabajo en equipo. El proyecto quedó dividido en módulos, cada uno a cargo de una etapa del proceso.

2.1. Gramática y manejo de errores sintácticos

La gramática (compilador.g4) define los tokens del lenguaje, las estructuras de control (if, while, for) y una jerarquía de reglas que respeta el orden de precedencia habitual de C para las expresiones aritmético-lógicas. También distingue el prototipo, la definición y la llamada de una función como casos separados, lo que facilita validarlos más adelante.

Para los errores sintácticos se reemplazó el manejador por defecto de ANTLR por uno propio (EscuchaSintactico.py), que traduce los mensajes técnicos del parser a avisos más claros —paréntesis o llaves faltantes, comas faltantes, y en particular el punto y coma, señalando la línea real donde falta.

2.2. Análisis semántico y tabla de símbolos

El análisis semántico (Escucha.py) usa el patrón Listener de ANTLR para recorrer el árbol y armar la Tabla de Símbolos a medida que se van reconociendo declaraciones, asignaciones y bloques. La tabla maneja los contextos como una pila: cada función o bloque anidado abre su

propio contexto, lo que permite detectar variables duplicadas, no declaradas o usadas sin inicializar.

2.3. Validación de return y de argumentos en llamadas

Sobre esa base se agregaron dos validaciones puntuales. Para el return, se busca la función que lo contiene y se verifica que el tipo de la expresión devuelta coincida con el tipo declarado por la función (permitiendo únicamente la conversión de int a float). Para las llamadas a funciones, se compara la cantidad y el tipo de los argumentos enviados contra los parámetros definidos en la función, reportando cualquier discrepancia sin cortar en el primer error, para que el estudiante vea todos los problemas de la llamada de una sola vez.

2.4. Generación y optimización de código intermedio

Una vez validado el árbol, un Visitor (Caminante.py) lo recorre y genera Código de Tres Direcciones: variables temporales para las expresiones, etiquetas para las estructuras de control, e instrucciones de invocación para las llamadas a funciones. Este paso solo se ejecuta si no hubo errores previos.

El código generado pasa luego por el Optimizador (Optimizador.py), que aplica de forma iterativa tres técnicas —plegado de constantes, propagación de copia y eliminación de código muerto— hasta que una pasada completa no introduce más cambios.

2.5. Controlador e integración

App.py coordina todo el flujo: lee el archivo fuente, instancia el lexer y el parser, y conecta los módulos de manejo de errores y análisis semántico. Según el resultado, genera un reporte de errores o bien los archivos TablaDeSimbolos.txt,CodigoIntermedio.txt y CodigoOptimizado.txt.

3. Conclusión

El trabajo permitió construir un compilador funcional para un subconjunto de C, cumpliendo con la consigna: detección de errores sintácticos y semánticos, manejo de contextos mediante tabla de símbolos, y generación y optimización de código intermedio a partir de una entrada válida.

Un punto que fue agregado luego de las correcciones realizadas a mi compañero fue el agregado de las validaciones de return y de argumentos en llamadas a funciones, que hacen al compilador más estricto a la hora de chequear que el uso de una función coincida con su definición.

Como limitaciones, el compilador no maneja punteros, structs ni memoria dinámica, y las optimizaciones implementadas son más básicas que las de un compilador profesional. Aun así, la organización modular del proyecto deja una base cómoda para futuras extensiones.